

Interfaces

Defining & Implementing Interfaces in Java

1. Concept Recap

An **interface** in Java is a blueprint for a class.

It defines a set of **abstract methods** (methods without implementation) that any implementing class **must** provide concrete definitions for.

With **Java 8**, interfaces can now also contain:

- **Default methods** (methods with a body, used to provide default behavior)
- **Static methods** (utility or helper methods within the interface)
- **Private methods** (Java 9 onwards, for internal interface logic reuse)

2. Syntax

```
public interface Vehicle {
    void start();
    void stop();
}

public class Car implements Vehicle {
    @Override
    public void start() {
        System.out.println("Car started");
    }

    @Override
    public void stop() {
        System.out.println("Car stopped");
    }
}
```

3. Best Practices

A. Naming Conventions

- Interface names should be **nouns** or **adjectives** (e.g., `Vehicle`, `Readable`, `Serializable`).
- Avoid using prefixes like `I` (common in C# but not in Java).

B. Purpose & Scope

- Use interfaces to define **contract-based design**.
- Keep them **focused** — follow the **Interface Segregation Principle** (ISP) from SOLID:
 - "Clients should not be forced to depend on methods they do not use."
- Avoid **"God interfaces"** — don't dump unrelated methods into one interface.

C. Method Design

- Define **only necessary methods** that all implementing classes are expected to use.
- Prefer **functional interfaces** (only 1 abstract method) if the intent is lambda usage.
- Add **JavaDoc** comments to explain expected behavior.

D. Use Default & Static Methods Wisely

- **Default methods**: Good for backward compatibility without breaking existing implementations.
- **Static methods**: Good for utility logic related to the interface's concept.

E. Implementation Best Practices

- Always use `@Override` when implementing methods — ensures compile-time safety.
- Avoid adding business logic **directly** in implementing class constructors; keep it in overridden methods.
- If multiple interfaces have default methods with the same signature, **resolve conflicts explicitly** in the implementing class.

4. Example with Best Practices

```
/**
 * Represents a generic payment processor.
 */
public interface PaymentProcessor {

    /**
     * Process a payment for the given amount.
     * @param amount the amount to process
     */
}
```

```

    */
    void processPayment(double amount);

    /**
     * Validate if payment method is available.
     * Default method to allow basic validation logic.
     */
    default boolean validate() {
        return true; // default: always valid
    }

    /**
     * Static method for common utility tasks.
     */
    static void logTransaction(String message) {
        System.out.println("[LOG] " + message);
    }
}

// Implementing class
public class CreditCardProcessor implements PaymentProcessor {

    @Override
    public void processPayment(double amount) {
        if (validate()) {
            System.out.println("Processing credit card payment of ₹" +
amount);
            PaymentProcessor.logTransaction("Credit Card payment
processed");
        }
    }
}

```

5. Common Mistakes to Avoid

- ❌ **Overloading interfaces with unrelated methods** — makes maintenance harder.
- ❌ **Using default methods for everything** — they should be used sparingly.
- ❌ **Breaking backward compatibility** — removing or changing interface method signatures can break existing code.
- ❌ **Mixing static data fields** — use constants in enums or a constants class instead.

6. When to Use

- To **decouple** code from specific implementations.
 - To **enable polymorphism** — same interface, multiple implementations.
 - For **dependency injection** — coding to an interface, not an implementation.
 - When **multiple inheritance** of behavior is needed (since Java supports multiple interfaces but not multiple class inheritance).
-

Functional Interfaces

Definition

A functional interface is any interface with **exactly one abstract method** (SAM — single abstract method). Java 8 introduces the `@FunctionalInterface` annotation that helps the compiler and documents intent.

Built-in common functional interfaces (java.util.function):

- `Supplier<T>` — supplies values (no args -> T)
- `Consumer<T>` — consumes values (T -> void)
- `Function<T, R>` — maps T to R
- `Predicate<T>` — boolean test
- `UnaryOperator<T>`, `BinaryOperator<T>` — specializations for same-type ops
- `BiFunction<T, U, R>`, `BiConsumer<T, U>` etc.

Comparator is also a functional interface: it has a single abstract method `compare(T, T)` despite having default methods.

Creating custom functional interfaces

```
@FunctionalInterface
public interface TriConsumer<A,B,C> {
    void accept(A a, B b, C c);

    // default or static methods are OK
    default void andThen(TriConsumer<A,B,C> after) {
        Objects.requireNonNull(after);
        return (a,b,c) -> {
```

```
        accept(a,b,c);
        after.accept(a,b,c);
    };
}
}
```

SAM-conversion

A lambda matches a functional interface — this is called SAM conversion. You can pass a lambda where a functional interface is expected.

Default and Static Methods in Interfaces

Motivation

Before Java 8 adding new methods to an existing interface would break all implementations. Default methods allow adding *optional* method implementations directly in interfaces to preserve backward compatibility.

default methods

- A **default** method has a body and can be overridden.
- If a class implements two interfaces providing the same default method signature, the implementing class **must** override the method to resolve the conflict.
- If a class extends a superclass that provides the same method, the *class/superclass* method takes precedence (class wins over interface).

Example:

```
public interface PaymentProcessor {
    void pay(Order o);

    default boolean refund(Order o, double amount) {
        // default refund implementation
        System.out.println("Default refund processing for order: " +
o.getId());
        return true;
    }
}
```

```

    }

    static String manName() { return "PaymentProcessor v1"; }
}

public class PaypalProcessor implements PaymentProcessor {
    @Override
    public void pay(Order o) { /* specific impl */ }

    // override default refund to implement provider-specific logic
    @Override
    public boolean refund(Order o, double amount) {
        // provider-specific logic
        return false;
    }
}

```

static methods in interfaces

- Belong to the interface type itself — not inherited by implementers.
- Useful for factory or helper methods tied to the interface contract.

```

public interface Converter<S,T> {
    T convert(S s);

    static Converter<String, Integer> parseInt() {
        return Integer::parseInt;
    }
}

```

Method resolution rules (summary)

- Class wins over interface.
 - Explicit override required if multiple interfaces provide same default method.
 - Use `InterfaceName.super.method()` to invoke a specific interface's default implementation.
-

Marker Interfaces

Definition

Marker interfaces are interfaces without methods used to mark classes for special treatment by the runtime or frameworks. Example: `Serializable`, `Cloneable`.

Use and tradeoffs

- Historically used for the JVM and libraries to establish a contract.
- **Drawback:** they mix metadata with type system; at runtime frameworks need `instanceof` checks.
- **Modern alternative:** use annotations for metadata (`@MyMarker`) — more flexible and expressive.

Example marker interface (rarely recommended now):

```
public interface SensitiveData { }
```



```
public class PatientRecord implements SensitiveData {  
    // fields  
}
```



```
// usage  
if (obj instanceof SensitiveData) {  
    // encrypt before persisting  
}
```

Prefer annotation approach when metadata suffices:

```
@Retention(RetentionPolicy.RUNTIME)  
@Target(ElementType.TYPE)  
public @interface Sensitive { }
```



```
@Sensitive  
public class PatientRecord { }
```
